افهم السؤال.
وبعدها خلّي الكود يقوله.

الشيت ده فيه جداول، حروف، شروط وهرم أرقام. كل سؤال هنا يبدأ من معناه، ويفكّك الفكرة اللي وراه، ثم يبني الكود بالتدريج. جرّب تتوقع اللي هيحصل قبل ما تكشف النتيجة.

الشيت الأصلي PDF Lecture 6 PDF الشرح والكود والصور يعملوا بدون إنترنت.

01

إحصاء وتتبع حالة

قراءات كتير، وإشارة واحدة توقفها

(1) Write a C++ program that reads an unspecified number of integers, determines how many positive and negative values have been read, and computes the total and average of the input values (not counting zeros). Your program ends with the input 0. Display the average as a floating-point number.

هو بيسأل عن إيه؟

السؤال مش بيقول هتدخل كام عدد. هتفضل تدخل أعداد صحيحة لحد ما تدخل صفر. المطلوب عدد الموجب، وعدد السالب، ومجموع القيم، ومتوسطها. الصفر هنا علامة نهاية، فلا يتحسب ضمن القراءات.

الفكرة قبل الكود

المتوسط هو المجموع مقسوم على عدد القيم. السالب يدخل في المجموع بإشارته، لكنه يزود عدد القراءات زي الموجب. لو قيمتين ألغوا بعض في المجموع، ده ما يلغيهمش من العدد. تخيّل تقرير منصة اختبار بيسجل انحرافات موجبة وسالبة عن القيمة المطلوبة؛ كل قراءة ليها مكان في التقرير.

اقرأ قيمة، وشوف اللي اتغيّر

ده تتبّع لنفس البرنامج بعد كل قراءة. التوقف عند الصفر لا يضيف قراءة جديدة.

3 | −1 | 4 | 1 | 0

3	1	4	7
عدد الموجب	عدد السالب	عدد القيم	المجموع

وصلت إشارة التوقف. المتوسط المسجل = 1.75. لو قسمنا int على int من غير تحويل: 1.

جرّب السلسلة الثانية: المتوسط فيها يكشف الفرق بين القسمة الصحيحة والقسمة بعد التحويل إلى float.

C++ اللي السؤال محتاجها

هنحتاج متغيرات منفصلة للموجب والسالب والمجموع، وكلها تبدأ قبل التكرار. حل الدكتور يستخدم `do-while`: نطلب الإدخال الأول، وبعدها نقدر نختبر هل وصلنا لإشارة التوقف. `if / else if` تصنّف المدخل، و`(float)` قبل القسمة يحافظ على الجزء الكسري. جسم do-while يتنفذ قبل اختبار الشرط؛ لذلك تحصل على أول إدخال حتى قبل معرفة قيمته. الفاصلة المنقوطة بعد while في النهاية جزء من تركيب do-while.

ابنِ الكود جزءًا جزءًا

جهّز الذاكرة

كل عدّاد له وظيفة مستقلة. بدء العدادات خارج التكرار يخليها تحتفظ باللي حصل قبل كده.

```
#include<iostream>
using namespace std;

int main()
{
    int number, positiveCount=0, negativeCount=0, nonZeroCount, totalSum=0;
    float average;
```

اقرأ وصنّف

الجسم يقرأ قيمة، يضيفها للمجموع، ويزود عدّاد الإشارة المناسب. إضافة الصفر للمجموع لا تغيّره، لكن الصفر لا يزود أي عدّاد. فحص cin يمنع الاستمرار لو قراءة العدد فشلت؛ return ينهي البرنامج بدل استخدام مدخل غير صالح.

```
do
    {
        cout << "Enter an integer, the input ends if it is 0: ";
        if (!(cin >> number)) return 0;
        totalSum = totalSum + number;
        if(number == 0)
        {
            cout << "You entered 0. The input is halted." << endl;
        }
        else if(number > 0)
        {
            positiveCount++;
        }
        else
            negativeCount++;
    }
    while(number!=0);
```

اعرف عدد القيم

نجمع عدد الموجب وعدد السالب بعد انتهاء الإدخال. ده المقام الصحيح للمتوسط.

```
nonZeroCount = positiveCount+negativeCount;
```

احسب المتوسط لو فيه قراءات

افحص المقام قبل القسمة. التحويل إلى float يحدث قبل العملية، مش بعد ما الجزء الكسري يضيع.

```
cout << "\nNumber of positive integers entered: " << positiveCount << endl;
    cout << "Number of negative integers entered: " << negativeCount << endl;
    cout << "Total sum of all integers: " << totalSum << endl;

    if (nonZeroCount == 0)
        cout << "No nonzero values; average is undefined." << endl;
    else
    {
        average = (float)totalSum/nonZeroCount;
        cout << "Average: " << average << endl;
    }
    return 0;
}
```

طريقة أخرى لنفس السؤال

تقدر تستخدم `while (true)` وتخرج بـ `break` عند الصفر. الاختلاف في مكان قرار التوقف؛ الحسابات والنتائج المسجلة هي نفسها. لازم يبقى فيه خروج واضح، لأن شرط while هنا يفضل صحيح.

```
#include<iostream>
using namespace std;

int main()
{
    int number, positiveCount=0, negativeCount=0, nonZeroCount, totalSum=0;
    float average;

    while (true)
    {
        cout << "Enter an integer, the input ends if it is 0: ";
        if (!(cin >> number)) return 0;
        totalSum = totalSum + number;
        if(number == 0)
        {
            cout << "You entered 0. The input is halted." << endl;
            break;
        }
        else if(number > 0)
        {
            positiveCount++;
        }
        else
            negativeCount++;
    }
    nonZeroCount = positiveCount+negativeCount;

    cout << "\nNumber of positive integers entered: " << positiveCount << endl;
    cout << "Number of negative integers entered: " << negativeCount << endl;
    cout << "Total sum of all integers: " << totalSum << endl;

    if (nonZeroCount == 0)
        cout << "No nonzero values; average is undefined." << endl;
    else
    {
        average = (float)totalSum/nonZeroCount;
        cout << "Average: " << average << endl;
    }
    return 0;
}
```

الطريقة البديلة اتجمّعت واتشغّلت على حالات الاختبار نفسها؛ الناتج المعروض مطابق.

شوف ناتج التشغيل الفعلي

المدخلات المرسلة إلى البرنامج

```
3 -1 4 0
```

stdout — رسائل البرنامج والنتائج، بدون صدى لوحة المفاتيح

```
Enter an integer, the input ends if it is 0: Enter an integer, the input ends if it is 0: Enter an integer,
the input ends if it is 0: Enter an integer, the input ends if it is 0: You entered 0. The input is halted.

Number of positive integers entered: 2
Number of negative integers entered: 1
Total sum of all integers: 6
Average: 2
```

ممكن تقع في ده

- عدّ كل محاولة إدخال يضيف الصفر ضمن البيانات. زوّد العدّاد للقيم المقبولة فقط.
- لو بدأت `totalSum = 0` جوه التكرار، هتمسح المجموع السابق في كل لفة.
- كتابة `average = totalSum / nonZeroCount` تقسم عددين صحيحين. نوع average لوحده لا يصلّح الجزء اللي ضاع.
- لو أول إدخال صفر، مفيش متوسط. النسخة دي تفحص الحالة بدل القسمة على صفر.
- عدد صحيح لا يعني سعة بلا حدود. القيم ومجموعها لازم يفضلوا داخل مدى int المستخدم.

جرّب تحسمها بنفسك

القيم الموجبة والسالبة ألغت بعض والمجموع بقى صفر. هل معنى كده مفيش قراءات؟

الإجابة: لا، العدد والمجموع حاجتين مختلفتين — بالضبط. ممكن المجموع يبقى صفر والعدد يفضل موجب.

> **لما تقارن بالمصدر:** حل المرجع لا يفحص حالة الصفر كأول إدخال. النسخة المشروحة هنا تضيف الفحص صراحة، وباقي التصنيف يتبع نفس الفكرة.

من millimeters إلى inches

(2) Write a C++ program that displays the following table (note that 1 $inch$ = 25.4 $millimeters$):

```
Millimeters     Inches
2               0.078
4               0.156
...
96              3.744
98              3.822
```

هو بيسأل عن إيه؟

اطبع جدول القيم اللي في العمود الأول، بدايةً من أول قيمة لحد آخر قيمة وبنفس الزيادة. قدّام كل طول بالـmillimeters اكتب الطول المقابل بالـinches. القيم تبدأ من 2 وتنتهي عند 98، بزيادة 2 كل مرة.

الفكرة قبل الكود

العلاقة المكتوبة في الشيت تقول إن البوصة الواحدة تساوي 25.4 millimeters. عشان تعرف الطول فيه كام بوصة، اقسمه على طول البوصة بالـmillimeters. فكر فيها كعدّ مجموعات متساوية، مش كقاعدة عمياء إن التحويل دايمًا ضرب أو قسمة.

طول واحد، اسمان للقياس

1 inch

25.4 mm

```
mm ÷ (mm / inch) = inch
```

الوحدة mm تختصر مع mm. المتبقي هو inch، وده يفسّر اتجاه القسمة.

2 mm ÷ 25.4 0.079 inch

نفس القيمة، شكل عرض مختلف

توقّع النتيجة، ثم اختَر طريقة العرض.

```
0.079
```

بدون precision، fixed تحسب الأرقام المعنوية: العد يبدأ عند أول رقم غير صفر. في 0.0787، الأرقام الثلاثة هي 7 ثم 8 ثم 7. مع fixed، تحسب الأرقام بعد العلامة العشرية. القيمة المخزنة نفسها لا تتغيّر.

عرض العمود يخص العنصر التالي

```
setw(6) << 2 << 4  → |     24|
left + setw(6)     → |2     4|
```

الحدّان | يوضحان المسافات في تجربة التنسيق. setw حجز عرضًا لأول قيمة فقط؛ القيمة التالية اتطبعت بعدها مباشرة. كرر setw قبل كل عمود. أما left وfixed وsetprecision فتستمر حتى تغيّرها.

C++ اللي السؤال محتاجها

`for` يجمع البداية والشرط والزيادة. ثابت التحويل من نوع `double` ، فالقسمة تفضل فيها كسور حتى لو المدخل `int` . `setw` يحجز عرضًا للعنصر التالي، و`left` يضبط المحاذاة. `fixed` مع `setprecision` يحددان شكل الجزء بعد العلامة العشرية. في for، البداية تتنفذ مرة واحدة؛ بعدها يتفحص الشرط قبل الجسم، ثم تتنفذ الزيادة، ثم يرجع الاختبار من جديد.

ابنِ الكود جزءًا جزءًا

ثابت التحويل

الاسم يربط الرقم بوحدته. الثابت مش بيتغيّر مع كل سطر.

```
#include<iostream>
#include <iomanip>
using namespace std;

int main() {
    const double MM_PER_INCH = 25.4;
```

العناوين قبل البيانات

العنوان يتطبع مرة واحدة. الطول والنتيجة هما اللي هيتكرروا.

```
cout << setw(15) << left << "Millimeters"
         << setw(15) << left << "Inches" << endl;
    cout << "----------------------" << endl;
```

حوّل قيمة واطبع سطرها

القسمة داخل التكرار، وزيادة المدخل في رأس for. التنسيق يقرّب العرض ولا يغيّر القيمة المخزنة.

```
for (int mm = 2; mm <= 98; mm=mm+2) {
        double inches = mm / MM_PER_INCH;
        cout << setw(15) << left << mm
             << setw(15) << left << fixed << setprecision(3) << inches << endl;
    }
    return 0;
}
```

طريقة أخرى لنفس السؤال

بدل القسمة على العامل، اضرب في مقلوبه `1.0 / MM_PER_INCH`. النتائج المطبوعة اتقارنت لكل الصفوف وطلعت متطابقة. احتفظ بالمقلوب الكامل؛ اختصاره إلى قيمة قليلة الخانات يغيّر النتائج.

```
#include<iostream>
#include <iomanip>
using namespace std;

int main() {
    const double MM_PER_INCH = 25.4;

    cout << setw(15) << left << "Millimeters"
         << setw(15) << left << "Inches" << endl;
    cout << "----------------------" << endl;

    for (int mm = 2; mm <= 98; mm=mm+2) {
        double inches = mm * (1.0 / MM_PER_INCH);
        cout << setw(15) << left << mm
             << setw(15) << left << fixed << setprecision(3) << inches << endl;
    }
    return 0;
}
```

الطريقة البديلة اتجمّعت واتشغّلت على حالات الاختبار نفسها؛ الناتج المعروض مطابق.

شوف ناتج التشغيل الفعلي

المدخلات المرسلة إلى البرنامج

```
لا يوجد إدخال
```

stdout — رسائل البرنامج والنتائج، بدون صدى لوحة المفاتيح

```
Millimeters    Inches
----------------------
2              0.079
4              0.157
6              0.236
8              0.315
10             0.394
12             0.472
14             0.551
16             0.630
18             0.709
20             0.787
22             0.866
24             0.945
26             1.024
28             1.102
30             1.181
32             1.260
34             1.339
36             1.417
38             1.496
40             1.575
```

```
42          1.654
44          1.732
46          1.811
48          1.890
50          1.969
52          2.047
54          2.126
56          2.205
58          2.283
60          2.362
62          2.441
64          2.520
66          2.598
68          2.677
70          2.756
72          2.835
74          2.913
76          2.992
78          3.071
80          3.150
82          3.228
84          3.307
86          3.386
88          3.465
90          3.543
92          3.622
94          3.701
96          3.780
98          3.858
```

ممكن تقع في ده

- ضرب millimeters في 25.4 يمشي في اتجاه التحويل العكسي. راجع وحدة المدخل والمطلوب.
- تخزين inches في `int` يتخلّص من الجزء الكسري.
- `setprecision(3)` من غير fixed مش معناه دايمًا ثلاثة أرقام بعد العلامة.
- `setw` لا يستمر لباقي السطر؛ كرره لكل عمود محتاج عرضًا محددًا.
- فاصلة منقوطة مباشرة بعد رأس for تجعل جسم الحلقة فارغًا؛ ضع الجسم المقصود داخل الأقواس المعقوفة.

جرّب تحسمها بنفسك

إيه اللي يحدد اتجاه الضرب أو القسمة؟

الإجابة: وحدة المدخل ووحدة المطلوب — اكتب معنى عامل التحويل بوحداته، وهتعرف العملية المطلوبة.

> **لما تقارن بالمصدر:** فيه فرق بين أرقام الجدول المصوّر والعلاقة المكتوبة. الشرح يتبع 25.4، زي حساب حل الدكتور. الكود هنا يعرض ثلاثة أرقام بعد العلامة مثل تنسيق الجدول، بينما المرجع يعرض أربعة. ده فرق معلَن، والصورة الأصلية موجودة فوق.

03

نسبة بين وحدتي طول

سطر تحويل واحد يتحول لجدول

(3) Write a program that displays the following table (note that $1\,meter = 3.280\,feet$):

Meters	Feet
1	3.280
2	6.560
...	
14	45.920
15	49.200

هو بيسأل عن إيه؟

لكل قيمة meters في الجدول، احسب عدد feet المقابل. المطلوب نفس مجال القيم ونفس ترتيب الصفوف، مع عناوين واضحة. مجال meters من 1 إلى 15 شاملًا الطرفين، والعامل المعطى 3.280 feet لكل meter.

الفكرة قبل الكود

كل meter يحتوي على عدد feet حسب العامل المعطى. لذلك تضرب طول meters في feet لكل meter. الرقم يزيد لأنك بتوصف نفس الطول بوحدة أصغر؛ الطول نفسه ما تغيّرش.

تغيير الوحدة لا يطوّل المسطرة



meters × (feet / meter) = feet

2 meters	× 3.280	6.560 feet

الصفر الأخير في 3.280 و6.560 جزء من شكل الجدول المطلوب. لذلك نحتاج fixed مع setprecision(3)، حتى لو القيمة يمكن كتابتها بعدد أقل من الخانات.

إيه اللي يتكرر؟

احسب طولًا واحدًا، اطبع العمودين، ثم انزل سطرًا. العناوين خارج التكرار، والتحويل داخله.

C++ اللي السؤال محتاجها

نحسب `Feet = M * M_Per_Feet` لكل قيمة M. اسم ثابت الدكتور ممكن يربكك، لكن دوره هنا feet لكل meter. استخدم `double` للناتج، واطبع `endl` بعد اكتمال السطر.

ابنِ الكود جزءًا جزءًا

حدد عامل التحويل

العامل علاقة بين الوحدتين، لا عدّاد للصفوف.

```
#include<iostream>
#include <iomanip>
using namespace std;

int main() {
    const double M_Per_Feet = 3.280;
```

اطبع رؤوس الأعمدة

البيانات تبدأ بعد العناوين والفاصل.

```
cout << setw(10) << left << "meters"
         << setw(10) << left << "Feet" << endl;
    cout << "----------------" << endl;
```

كرر حساب السطر

تتحرك M في المجال المطلوب. حاصل الضرب يعاد حسابه لنفس الطول الحالي ثم يُطبع.

```
for (int M = 1; M <= 15; M++) {
        double Feet = M * M_Per_Feet;
        cout << setw(10) << left << M
             << setw(10) << left << fixed << setprecision(3) << Feet << endl;
    }
    return 0;
}
```

طريقة أخرى لنفس السؤال

نفس الجدول ينفع بـ `while` : البداية قبل التكرار، الاختبار قبل كل سطر، والزيادة في نهاية الجسم. دي إعادة ترتيب لأجزاء for، مش قانون تحويل جديد. نسيان الزيادة يكرر نفس السطر بلا نهاية.

```
#include<iostream>
#include <iomanip>
using namespace std;

int main() {
    const double M_Per_Feet = 3.280;

    cout << setw(10) << left << "meters"
         << setw(10) << left << "Feet" << endl;
    cout << "----------------" << endl;

    int M = 1;
    while (M <= 15) {
        double Feet = M * M_Per_Feet;
        cout << setw(10) << left << M
             << setw(10) << left << fixed << setprecision(3) << Feet << endl;
        M++;
    }
    return 0;
}
```

الطريقة البديلة اتجمّعت واتشغّلت على حالات الاختبار نفسها؛ الناتج المعروض مطابق.

شوف ناتج التشغيل الفعلي

المدخلات المرسلة إلى البرنامج

```
لا يوجد إدخال
```

stdout — رسائل البرنامج والنتائج، بدون صدى لوحة المفاتيح

```
meters    Feet
----------------
1         3.280
2         6.560
3         9.840
4         13.120
```

```
5         16.400
6         19.680
7         22.960
8         26.240
9         29.520
10        32.800
11        36.080
12        39.360
13        42.640
14        45.920
15        49.200
```

ممكن تقع في ده

- اعتبار الأصفار الأخيرة خطأ: `3.280` شكل عرض مقصود، مش قيمة مختلفة عن 3.28.
- طباعة العنوان داخل التكرار تجعله يظهر بين الصفوف.
- وضع `endl` خارج التكرار فقط يضع الصفوف كلها على سطر واحد.
- إعادة استخدام عامل التحويل بالعكس تخلط feet→meters مع meters→feet.

جرّب تحسمها بنفسك

ليه القيمة العددية بالـfeet أكبر لنفس الطول؟

الإجابة: لأن الوحدة أصغر — نحتاج وحدات أكثر لما تكون الوحدة أصغر.

> **لما تقارن بالمصدر:** الحساب والمجال متفقان مع المرجع. المرجع يعرض رقمين بعد العلامة، بينما صورة الشيت تعرض ثلاثة؛ النسخة هنا تطابق تنسيق الصورة.

تحويل في اتجاهين

04

جدولين جنب بعض، ومدخلين مختلفين

(4) Write a program that displays the following tables side by side (note that $1\ inch = 25.4\ millimeters$):

```
Millimeters     Inches   |   Inches     Millimeters
2               0.078    |   1          25.641
4               0.156    |   2          51.282
...
98              3.822    |   49         1256.410
100             3.900    |   50         1282.051
```

هو بيسأل عن إيه؟

السطر فيه تحويلان مستقلان. الشمال يبدأ بقيمة millimeters، واليمين يبدأ بقيمة inches. اعرض الزوجين جنب بعض، مع زيادة كل عمود مصدر بالطريقة اللي في الشيت. الشمال من 2 إلى 100 millimeters بزيادة 2؛ اليمين من 1 إلى 50 inches بزيادة 1.

الفكرة قبل الكود

العلاقة نفسها تُقرأ في اتجاهين. من millimeters إلى inches نقسم على 25.4. من inches إلى millimeters نضرب فيه. اشتراك الجدولين في سطر واحد لا يعني إن ناتج الشمال يدخل حساب اليمين.

الصف يحتوي رحلتين مستقلتين

العمود الثالث مدخل جديد؛ مش ناتج تحويل العمود الأول.

mm	÷ 25.4 → inch	inch	× 25.4 → mm
2	0.079	1	25.400

```
mm += 2     |     inches++
```

العلاقة بين الوحدات عكسية، لكن عمودي البداية يتقدّموا بزيادتين مختلفتين. لو استخدمت ناتج الجهة الأولى كبداية للجهة الثانية، هتبني جدولًا آخر.

C++ اللي السؤال محتاجها

المرجع يستخدم j للـmillimeters و`j/2` للـinches. القسمة الصحيحة هنا مقصودة لأن j يمر بقيم زوجية. في الاتجاه العكسي، `inch / Inch_Per_mm` قسمة على المقلوب، وتساوي الضرب في العامل الأصلي.

ابنِ الكود جزءًا جزءًا

جهّز العامل ومقلوبه

وجود ثابتين لا يعني وجود قانونين مختلفين.

```
#include<iostream>
#include <iomanip>
using namespace std;

int main() {
    const double mm_Per_Inch = 25.4;
    const double Inch_Per_mm = 1/25.4;
```

كوّن المدخلين

كل لفة تختار موضعًا من كل جدول. j/2 يحدد مصدر اليمين من رقم الصف.

```
cout << setw(12) << left << "mms"
         << setw(12) << left << "Inches      |";
    cout << setw(12) << left << "Inches"
         << setw(12) << left << "mms" << endl;
    cout << "-----------------------------------------------------" << endl;

    int mms, inch;
    double Inches, milli;

    for (int j = 2; j <= 100; j=j+2) {
        mms = j;
        inch = j/2;
```

احسب الاتجاهين

كل تحويل يستخدم مدخله الخاص. راقب الوحدات قبل النظر للتنسيق.

```
Inches = mms / mm_Per_Inch;
        milli = inch / Inch_Per_mm;
```

اطبع الصف كاملًا

ضع الفاصل بين الجدولين، والعرض المناسب أمام كل قيمة، ثم انتقل لسطر جديد.

```
cout << setw(12) << left << mms
             << setw(12) << left << fixed << setprecision(3) << Inches << "| ";

        cout << setw(12) << left << inch
             << setw(12) << left << fixed << setprecision(3) << milli << endl;
    }

    return 0;
}
```

طريقة أخرى لنفس السؤال

اكتب الاتجاه العكسي مباشرةً `milli = inch * mm_Per_Inch` بدل القسمة على المقلوب. ده أوضح لو المقلوب بيشتتك. تمت مقارنة كل الصفوف مع النسخة الأساسية.

```
#include<iostream>
#include <iomanip>
using namespace std;

int main() {
    const double mm_Per_Inch = 25.4;

    cout << setw(12) << left << "mms"
         << setw(12) << left << "Inches      |";
    cout << setw(12) << left << "Inches"
         << setw(12) << left << "mms" << endl;
    cout << "-----------------------------------------------" << endl;

    int mms, inch;
    double Inches, milli;

    for (int j = 2; j <= 100; j=j+2) {
        mms = j;
        inch = j/2;
        Inches = mms / mm_Per_Inch;
        milli = inch * mm_Per_Inch;

        cout << setw(12) << left << mms
             << setw(12) << left << fixed << setprecision(3) << Inches << "| ";

        cout << setw(12) << left << inch
             << setw(12) << left << fixed << setprecision(3) << milli << endl;
    }

    return 0;
}
```

الطريقة البديلة اتجمّعت واتشغّلت على حالات الاختبار نفسها؛ الناتج المعروض مطابق.

شوف ناتج التشغيل الفعلي

المدخلات المرسلة إلى البرنامج

```
لا يوجد إدخال
```

stdout — رسائل البرنامج والنتائج، بدون صدى لوحة المفاتيح

```
mms         Inches      |Inches      mms
-----------------------------------------------
2           0.079       | 1           25.400
4           0.157       | 2           50.800
6           0.236       | 3           76.200
8           0.315       | 4           101.600
10          0.394       | 5           127.000
12          0.472       | 6           152.400
14          0.551       | 7           177.800
```

```
16          0.630       | 8           203.200
18          0.709       | 9           228.600
20          0.787       | 10          254.000
22          0.866       | 11          279.400
24          0.945       | 12          304.800
26          1.024       | 13          330.200
28          1.102       | 14          355.600
30          1.181       | 15          381.000
32          1.260       | 16          406.400
34          1.339       | 17          431.800
36          1.417       | 18          457.200
38          1.496       | 19          482.600
40          1.575       | 20          508.000
42          1.654       | 21          533.400
44          1.732       | 22          558.800
46          1.811       | 23          584.200
48          1.890       | 24          609.600
50          1.969       | 25          635.000
52          2.047       | 26          660.400
54          2.126       | 27          685.800
56          2.205       | 28          711.200
58          2.283       | 29          736.600
60          2.362       | 30          762.000
62          2.441       | 31          787.400
64          2.520       | 32          812.800
66          2.598       | 33          838.200
68          2.677       | 34          863.600
70          2.756       | 35          889.000
72          2.835       | 36          914.400
74          2.913       | 37          939.800
76          2.992       | 38          965.200
78          3.071       | 39          990.600
80          3.150       | 40          1016.000
82          3.228       | 41          1041.400
84          3.307       | 42          1066.800
86          3.386       | 43          1092.200
88          3.465       | 44          1117.600
90          3.543       | 45          1143.000
92          3.622       | 46          1168.400
94          3.701       | 47          1193.800
96          3.780       | 48          1219.200
98          3.858       | 49          1244.600
100         3.937       | 50          1270.000
```

ممكن تقع في ده

- استخدام ناتج inches من الشمال كمدخل اليمين يربط جدولين مستقلين بالخطأ.
- حل الجدولين بتكرار داخل تكرار يطبع تركيبات كثيرة بدل صفين متجاورين.
- القول إن القسمة دائمًا غلط في inches→mm يتجاهل إن المقسوم عليه قد يكون المقلوب.
- تطبيق setw مرة واحدة لا يحدد عرض كل الأعمدة.

جرّب تحسمها بنفسك

مصدر جدول اليمين هو...

الإجابة: القيمة المحددة للـinches في نفس الصف — صح؛ له مدخل مستقل ومجاله الخاص.

> **لما تقارن بالمصدر:** الأرقام المصوّرة لا تتفق مع عامل 25.4 المكتوب. الحساب هنا يتبع العامل وحل المرجع، مع إبقاء الصورة الأصلية والملاحظة ظاهرين.

05

النسبة والمقلوب

بدّل الوحدة، وخلّي كل عمود ماشي بخطوته

(5) Write a program that displays the following tables side by side (note that $1\ meter = 3.280\ feet$):

Meters	Feet		Feet	Meters
1	3.280	|	3	0.915
2	6.560	|	6	1.829
...				
14	45.920	|	42	12.805
15	49.200	|	45	13.720

هو بيسأل عن إيه؟

اطبع meters→feet في الشمال، وfeet→meters في اليمين. المصدر في الشمال يتحرك بوحدة واحدة، والمصدر في اليمين يتحرك بخطوة مختلفة كما يظهر في السؤال. قيم meters من 1 إلى 15 بزيادة 1. قيم feet من 3 إلى 45 بزيادة 3.

الفكرة قبل الكود

التحويل العكسي يرجع لنفس العلاقة الأصلية. ضرب meters في feet لكل meter يعطي feet. قسمة feet على نفس العامل تعطي meters. ما تستخدمش نتيجة مقرّبة من جدول كأنها ثابت تحويل.

الضرب في اتجاه، والقسمة في العكس

meters	× 3.280 →	feet

feet	÷ 3.280 →	meters

m	ft	ft	m
1	3.280	3	0.915

مصدر الجهة اليسار يزيد meter واحدًا في كل صف. مصدر الجهة اليمين يزيد ثلاثة feet. عدّاد الصف يزامن الجهتين، لكنه ليس عامل التحويل.

C++ اللي السؤال محتاجها

`Meters = j` و`Feet = 3*j` يجهزان المصدرين. المرجع يحسب `Met = Feet / feet_Per_meter`، ويحسب الاتجاه الآخر بقسمة على المقلوب. `fixed` و`setprecision` يستمران حتى تغيّرهما.

ابنِ الكود جزءًا جزءًا

اكتب العلاقة ومقلوبها

خلي المعنى مرتبطًا بالوحدات، لأن أسماء المتغيرات وحدها ممكن تتفهم بالعكس.

```
#include<iostream>
#include <iomanip>
using namespace std;

int main() {
    const double feet_Per_meter = 3.28;
    const double meter_Per_feet = 1/3.28;
```

كوّن المصدرين

الصف نفسه يحمل طولين مختلفين، واحد بالمتر وواحد بالقدم.

```
cout << setw(12) << left << "Meters"
         << setw(12) << left << "Feet        |";
    cout << setw(12) << left << "Feet"
         << setw(12) << left << "Meters" << endl;
    cout << "-----------------------------------------------------" << endl;

    int Meters, Feet;
    double Met, Ft;

    for (int j = 1; j <= 15; j++) {
        Meters = j;
        Feet = 3*j;
```

حوّل ونسّق

احسب كل ناتج من مصدره، ثم اطبع الأعمدة الأربعة. التنسيق لا يدخل في المعادلة.

```
Met = Feet / feet_Per_meter;
        Ft = Meters / meter_Per_feet;

        cout << setw(12) << left << Meters
             << setw(12) << left << fixed << setprecision(3) << Ft << "| ";

        cout << setw(12) << left << Feet
             << setw(12) << left << fixed << setprecision(3) << Met << endl;
    }

    return 0;
}
```

طريقة أخرى لنفس السؤال

للاتجاه meters→feet، استخدم `Ft = Meters * feet_Per_meter` بدل القسمة على meter_Per_feet. العمليتان متكافئتان هنا، والصفوف المسجلة متطابقة.

```
#include<iostream>
#include <iomanip>
using namespace std;

int main() {
    const double feet_Per_meter = 3.28;

    cout << setw(12) << left << "Meters"
         << setw(12) << left << "Feet        |";
    cout << setw(12) << left << "Feet"
         << setw(12) << left << "Meters" << endl;
    cout << "-----------------------------------------------" << endl;

    int Meters, Feet;
    double Met, Ft;

    for (int j = 1; j <= 15; j++) {
        Meters = j;
        Feet = 3*j;
        Met = Feet / feet_Per_meter;
        Ft = Meters * feet_Per_meter;

        cout << setw(12) << left << Meters
             << setw(12) << left << fixed << setprecision(3) << Ft << "| ";

        cout << setw(12) << left << Feet
             << setw(12) << left << fixed << setprecision(3) << Met << endl;
    }

    return 0;
}
```

الطريقة البديلة اتجمّعت واتشغّلت على حالات الاختبار نفسها؛ الناتج المعروض مطابق.

شوف ناتج التشغيل الفعلي

المدخلات المرسلة إلى البرنامج

```
لا يوجد إدخال
```

stdout — رسائل البرنامج والنتائج، بدون صدى لوحة المفاتيح

```
Meters      Feet        |Feet       Meters
-----------------------------------------------
1           3.280       | 3          0.915
2           6.560       | 6          1.829
3           9.840       | 9          2.744
4           13.120      | 12         3.659
5           16.400      | 15         4.573
6           19.680      | 18         5.488
7           22.960      | 21         6.402
8           26.240      | 24         7.317
9           29.520      | 27         8.232
10          32.800      | 30         9.146
11          36.080      | 33         10.061
12          39.360      | 36         10.976
13          42.640      | 39         11.890
14          45.920      | 42         12.805
15          49.200      | 45         13.720
```

ممكن تقع في ده

- زيادة المصدرين بنفس المقدار لا تطابق السؤال.
- اعتبار الرقم في العمود الثاني مصدرًا للعمود الثالث يربط جدولين مستقلين.
- تخزين Met أو Ft في int يخسر الكسور.
- تغيير الثابت كل مرة من نتيجة مقرّبة يقلل الدقة بلا داعٍ.

جرّب تحسمها بنفسك

Feet في الجدول اليمين بتتحسب من إيه؟

الإجابة: من رقم الصف وخطوة المصدر المطلوبة — كل جدول له مصدره.

> **لما تقارن بالمصدر:** مجال القيم والعامل والنتائج متفقة مع الشيت وحل المرجع. الاختلافات المعروضة هنا طرق كتابة مكافئة.

افهم النسبة، وبعدها سلّم الدالة زاويتها

(6) Write a C++ program to print the following table to display the tan and cot values of degrees from 0 to 60 with increments of 10 degrees. Round the value to keep four digits after the decimal point.

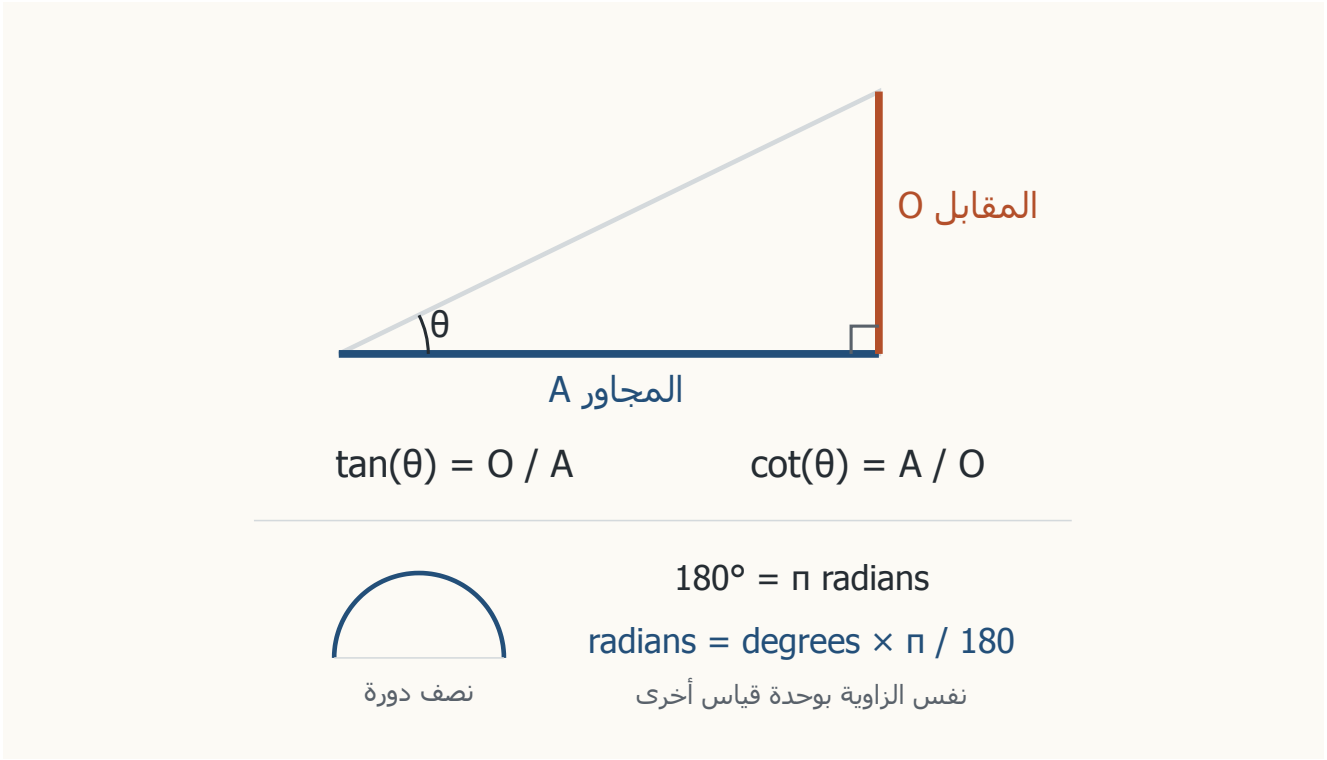
```
Degree      Sin/Cos      Cos/Sin
10          0.0000       inf
20          0.1736       5.6713
...
50          1.1918       0.8391
60          1.7320       0.5774
```

هو بيسأل عن إيه؟

اطبع زاوية كل صف وقيمتي tan وcot، من بداية المجال المكتوب إلى نهايته وبالخطوة المطلوبة. اعرض أربعة أرقام بعد العلامة العشرية. المجال المكتوب من 0 إلى 60 degrees، بزيادة 10 degrees.

الفكرة قبل الكود

في مثلث قائم وبالنسبة لزاوية محددة، tan هي طول المقابل مقسومًا على المجاور. cot هي النسبة بالعكس. وباستخدام الدوال: tan = sin/cos وcot = cos/sin. دوال C++ تستقبل radians، بينما السؤال يعطي degrees، فلازم تغيّر وحدة الزاوية قبل الاستدعاء.



حدّد الزاوية الأول، وبعدها سمّي المقابل والمجاور بالنسبة لها. تبديل المقام والبسط يبدّل tan وcot. وتحويل degrees إلى radians يغيّر وحدة القياس، مش الزاوية نفسها. المثلث توضيحي ولا يمثل زاوية رقمية بعينها.

الجدول المشغّل أسفل الشرح يعرض الزوايا المطلوبة وقيمها. عند الصفر لا توجد قيمة cot معرفة؛ كلمة undefined مقصودة وليست خطأ طباعة.

C++ اللي السؤال محتاجها

نحصل على PI بـ `acos(-1.0)` من cmath، ثم `rad = j*PI/180` . اضرب القيمة في عامل التحويل قبل تنادي tan. عند الزاوية صفر، tan تساوي صفرًا، فلا تُجر قسمة الواحد عليها.

ابنِ الكود جزءًا جزءًا

جهّز الدوال وPI

acos ترجع الزاوية التي cosine لها يساوي المدخل. القيمة المستخدمة هنا تعطي PI بوحدة radians، من غير الاعتماد على M_PI. لأن cos(π) = −1، فإن acos(−1) تعطي π.

```
#include<iostream>
#include <iomanip>
#include <cmath>
using namespace std;

int main() {
    const double PI = acos(-1.0);
```

غيّر وحدة الزاوية

الزاوية نفسها ثابتة؛ اللي بيتغيّر هو الرقم والوحدة المستخدمة لقياسها.

```
double sin_per_cos, rad;

    cout << setw(10) << left << "angle"
         << setw(10) << left << "sin/cos" << setw(10) << left << "cos/sin" << endl;

    for(int j=0;j<=60;j=j+10)
    {
        rad=j*PI/180;
```

احسب وافحص المقام

الدالة تعطي tan، والفرع يمنع قسمة cot عند الصفر. التنسيق يخص العرض النهائي فقط.

```
sin_per_cos=tan(rad);
        cout << setw(10) << left << j
             << setw(10) << left << fixed << setprecision(4) << sin_per_cos;

        if (j == 0) cout << "undefined" << endl;
        else cout << fixed << setprecision(4) << 1/tan(rad) << endl;
    }
    return 0;
}
```

طريقة أخرى لنفس السؤال

احسب `sin(rad)/cos(rad)` و `cos(rad)/sin(rad)` بدل tan ومقلوبها. نفس النسب، مع نفس فحص الصفر. القيم المعروضة بأربعة أرقام بعد العلامة اتقارنت وطلعت متطابقة.

```
#include<iostream>
#include <iomanip>
#include <cmath>
using namespace std;

int main() {
    const double PI = acos(-1.0);
    double sin_per_cos, rad;

    cout << setw(10) << left << "angle"
         << setw(10) << left << "sin/cos" << setw(10) << left << "cos/sin" << endl;

    for(int j=0;j<=60;j=j+10)
    {
        rad=j*PI/180;
        sin_per_cos=sin(rad)/cos(rad);
        cout << setw(10) << left << j
             << setw(10) << left << fixed << setprecision(4) << sin_per_cos;

        if (j == 0) cout << "undefined" << endl;
        else cout << fixed << setprecision(4) << cos(rad)/sin(rad) << endl;
    }
    return 0;
}
```

الطريقة البديلة اتجمّعت واتشغّلت على حالات الاختبار نفسها؛ الناتج المعروض مطابق.

شوف ناتج التشغيل الفعلي

المدخلات المرسلة إلى البرنامج

```
لا يوجد إدخال
```

stdout — رسائل البرنامج والنتائج، بدون صدى لوحة المفاتيح

```
angle     sin/cos   cos/sin
0         0.0000    undefined
10        0.1763    5.6713
20        0.3640    2.7475
30        0.5774    1.7321
40        0.8391    1.1918
50        1.1918    0.8391
60        1.7321    0.5774
```

ممكن تقع في ده

- كتابة `tan(j)` مباشرةً تجعل الدالة تفهم j كـradians، لا degrees.
- `1/tan` مش نفس inverse tangent. الدالة acos أو atan تعيد زاوية؛ المقلوب يعكس نسبة.
- الرمز inf الذي يطبعه نظام معين لا يجعل cot عند الصفر قيمة رياضية محددة.
- M_PI قد يكون متاحًا على جهاز وغير متاح على آخر؛ ليس اسمًا مضمونًا في C++98.

جرّب تحسمها بنفسك

إيه اللي يتغير عند التحويل من degrees إلى radians؟

الإجابة: وحدة قياس نفس الزاوية — نفس الدوران له رقم مختلف لأن وحدة القياس مختلفة.

> **لما تقارن بالمصدر:** الصورة فيها تسميات صفوف وقيم لا تتفق مع بعضها. الشرح يتبع مجال الزوايا المكتوب والقيم المنفذة. المرجع يقسم عند الصفر ويظهر inf على الجهاز المختبر؛ هنا نكتب undefined قبل القسمة. قيمة tan عند النهاية تتبع التقريب المسجل، لا الرقم المصور.

الدالة الأسية

07

جدول e مرفوعة للأس

(7) Write a C++ program to print the following table using the *exp* function:

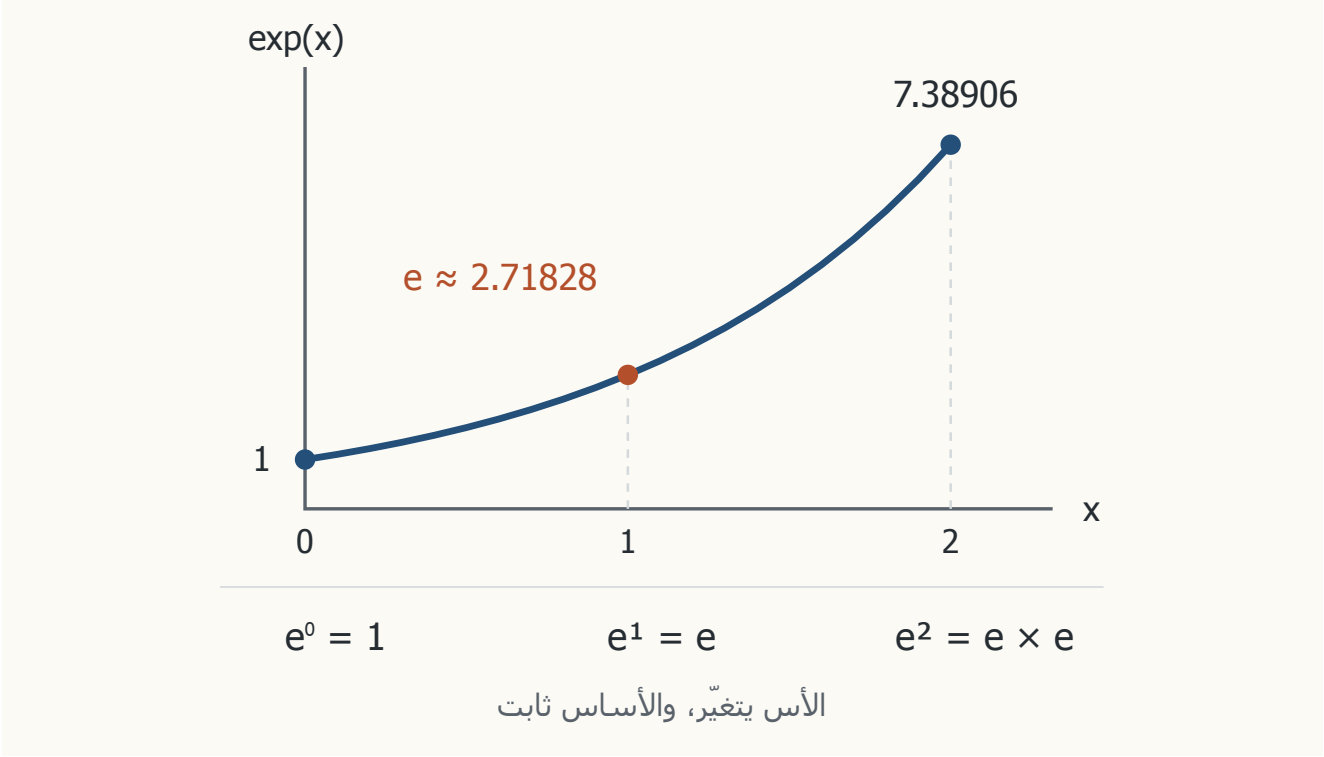
Number	Exponent
0	1
1	2.71828
...	
9	8103.08
10	22026.5

هو بيسأل عن إيه؟

السؤال يطلب استخدام exp لحساب قيمة الدالة لكل مدخل في الجدول. لا يطلب منك كتابة متسلسلة لحساب e أو إعادة تنفيذ الدالة من الداخل. قيم x هي الأعداد الصحيحة من 0 إلى 10، شاملًا الطرفين.

الفكرة قبل الكود

`exp(x)` معناها e مرفوعة للأس x. الأس يحدد مقدار النمو؛ ليس عدد الأرقام التي تريد طباعتها. الدوال الأسية تظهر في نماذج النمو والاضمحلال، لكن هذا السؤال نفسه جدول رياضي بلا وحدة فيزيائية محددة.



الأس يتغيّر، والأساس ثابت

الأساس e ثابت، وقيمته تظهر عند تشغيل exp(1). في الأسس الصحيحة، كل زيادة في الأس تضيف عامل ضرب في e؛ مش إضافة عدد ثابت للناتج. النقط والأرقام مأخوذة من تشغيل الكود، والأرقام المكتوبة مقرّبة للعرض.

$$\exp(x) = e^x$$

المنحنى يعرض أول قيم من التشغيل لتلاحظ الفكرة؛ الجدول الكامل يصل إلى نهاية المجال المطلوب. في C++ العامل ^ لا يحسب الأسّ.

C++ اللي السؤال محتاجها

ضمّن cmath واستدعِ `exp(j)` داخل التكرار، وخزّن الناتج في double. عدّاد j يمكن أن يكون int؛ وسيُحوَّل لمدخل الدالة المناسب. العرض الافتراضي هنا يتبع الأرقام المهمة، لذلك عدد الأرقام بعد العلامة لا يظل ثابتًا.

ابنِ الكود جزءًا جزءًا

جهّز exp والمتغيرات

اسم الدالة متاح من cmath. كتابة اسمها صح لا تعوّض غياب الـheader.

```
#include<iostream>
#include <iomanip>
#include <cmath>
```

اختَر المدخل المتكرر

كل صف يحسب الدالة من j الحالي، وليس من الناتج السابق.

```
using namespace std;

int main() {
    double Exponent;

    cout << setw(10) << left << "Number(x)"
         << setw(10) << left << "Exponent(e^x)" << endl;

    for(int j=0;j<=10;j++)
    {
        Exponent = exp(j);
```

اطبع المدخل والناتج

خلي المتغيرين ظاهرين جنب بعض عشان تعرف كل قيمة خرجت من أي مدخل.

```
cout << setw(10) << left << j << setw(11) << left << Exponent <<endl;
    }
    return 0;
}
```

طريقة أخرى لنفس السؤال

استخدم while بدل for، مع تهيئة j قبله وزيادته بعد الصف. ده يغير تركيب التكرار فقط. كل قيم الجدول وعرضها تطابقت في الاختبار.

```
#include<iostream>
#include <iomanip>
#include <cmath>
using namespace std;

int main() {
    double Exponent;

    cout << setw(10) << left << "Number(x)"
         << setw(10) << left << "Exponent(e^x)" << endl;

    int j = 0;
    while (j <= 10)
    {
        Exponent = exp(j);
        cout << setw(10) << left << j << setw(11) << left << Exponent <<endl;
        j++;
    }
    return 0;
}
```

الطريقة البديلة اتجمّعت واتشغّلت على حالات الاختبار نفسها؛ الناتج المعروض مطابق.

شوف ناتج التشغيل الفعلي

المدخلات المرسلة إلى البرنامج

```
لا يوجد إدخال
```

stdout — رسائل البرنامج والنتائج، بدون صدى لوحة المفاتيح

```
Number(x) Exponent(e^x)
0         1
1         2.71828
2         7.38906
3         20.0855
4         54.5982
5         148.413
6         403.429
7         1096.63
8         2980.96
9         8103.08
10        22026.5
```

ممكن تقع في ده

- `exp(x)` لا تعني `x*e` ولا `pow(x,e)` .
- العلامة `^` في C++ عملية على البتات، مش رفع للقوة.
- إضافة fixed لأنك استخدمته في Q6 تغيّر شكل جدول Q7؛ راجع المطلوب لكل سؤال.
- تخزين Exponent في int يتخلص من الجزء الكسري.

جرّب تحسمها بنفسك

x في exp(x) بتحدد إيه؟

الإجابة: الأس الذي نرفع e إليه — ده مدخل الدالة الرياضي.

> **لما تقارن بالمصدر:** المجال ودالة exp وعرض المرجع متفقة مع السؤال. القيم المتغيرة في عدد الخانات هنا ليست خطأ في الدالة.

القسمة والباقي والمنطق

08

فلتر الأرقام، وبعدها عدّ اللي اتطبع

(8) Write a C++ program that displays all the numbers from 1 to 500, 5 per line, that are divisible by 3 and 4. Numbers are separated by exactly one space.

هو بيسأل عن إيه؟

مرّ على المجال المطلوب، واطبع الأرقام التي تقبل القسمة على 3 وعلى 4 معًا. المجال من 1 إلى 500. كل سطر فيه خمس قيم مقبولة، وبين القيم مسافة واحدة.

الفكرة قبل الكود

العدد يقبل القسمة إذا كان باقي القسمة صفرًا. كلمة «و» معناها إن الاختبارين لازم ينجحوا في نفس الوقت. المجال كله مرشحون، لكن السطر لا يمتلئ إلا بالقيم المقبولة.

البوابة لا تعدّ المرشحين

توقّع قبول المرشح قبل تحريك الاختيار. كل البواقي والحالات هنا من برنامج C++ مشغّل.

60

`i % 4`
`0`
المطلوب = 0
	&&	`i % 3`
		`0`
		المطلوب = 0

مقبول: الشرطان متحققان. زوّد عدّاد المطبوع.

عدد ما طُبع في هذا التتبّع: **5**

12	24	36	48	60

التتبّع جزء من المجال لتشوف القرار. التشغيل الكامل موجود تحت الكود. السطر الجديد يتبع عدّاد المطبوع، لا قيمة المرشح.

C++ اللي السؤال محتاجها

استخدم `%` للباقي، و`&&` للجمع بين الشرطين. افصل عدّاد المرشح عن عدّاد المطبوع. مسافة الفصل تتطبع بين العناصر فقط، وبعد اكتمال المجموعة اطبع newline.

ابنِ الكود جزءًا جزءًا

جهّز عدّاد المطبوع

عدد المرشحين لا يساوي عدد المقبولين. `numbers_per_line` يزيد وقت الطباعة فقط.

```
#include<iostream>
#include <iomanip>
using namespace std;

int main() {
    int numbers_per_line = 0;
```

اختبر الشرطين

الفرع ينفذ فقط إذا نجح الاختباران معًا.

```
for (int i = 1; i <= 500; ++i)
    {

        if ((i % 3 == 0) && (i % 4 == 0))
```

املأ سطر الإخراج

أضف مسافة عندما يوجد عنصر سابق في السطر، ثم القيمة والعدّاد. أخيرًا اقفل السطر الكامل أو الجزئي.

```
{

            if (numbers_per_line % 5 != 0) cout << " ";
            cout << i;
            numbers_per_line++;
            if (numbers_per_line % 5 == 0) cout << endl;
        }
    }
    if (numbers_per_line % 5 != 0) cout << endl;

    return 0;
}
```

طريقة أخرى لنفس السؤال

القيم المقبولة هي مضاعفات أقل مضاعف مشترك للمقسوم عليهما: أصغر عدد موجب يقبل القسمة على الاثنين. لذلك شرط المرجع `i % 12 == 0` مكافئ هنا. حافظ على عدّاد المطبوع حتى لو اختصرت شرط القبول.

```
#include<iostream>
#include <iomanip>
using namespace std;

int main() {
    int numbers_per_line = 0;

    for (int i = 1; i <= 500; ++i)
    {

        if (i % 12 == 0)
        {

            if (numbers_per_line % 5 != 0) cout << " ";
            cout << i;
            numbers_per_line++;
            if (numbers_per_line % 5 == 0) cout << endl;
        }
    }
    if (numbers_per_line % 5 != 0) cout << endl;

    return 0;
}
```

الطريقة البديلة اتجمّعت واتشغّلت على حالات الاختبار نفسها؛ الناتج المعروض مطابق.

شوف ناتج التشغيل الفعلي

المدخلات المرسلة إلى البرنامج

```
لا يوجد إدخال
```

stdout — رسائل البرنامج والنتائج، بدون صدى لوحة المفاتيح

```
12 24 36 48 60
72 84 96 108 120
132 144 156 168 180
192 204 216 228 240
252 264 276 288 300
312 324 336 348 360
372 384 396 408 420
432 444 456 468 480
492
```

ممكن تقع في ده

- `||` يقبل النجاح في أي اختبار، وليس الاثنين معًا.
- وضع `numbers_per_line++` خارج فرع القبول يعد المرشحين المرفوضين.
- اختبار i % 5 لتحديد نهاية السطر يعد مواضع المرشحين، لا عدد ما طُبع.
- setw مع مسافة إضافية يزيد الفاصل عن مسافة واحدة.

جرّب تحسمها بنفسك

السطر الجديد يتحدد من...

الإجابة: عدد القيم التي طُبعت فعلًا — المطبوع هو اللي يحتل خانات السطر.

> **لما تقارن بالمصدر:** شرط المرجع المختصر صحيح، لكن padding بـsetw والمسافات الأخيرة لا يطابقان شرط المسافة الواحدة. النسخة هنا تفصل العناصر بمسافة واحدة وتختم السطر الجزئي.

09

شرطان لازم يتحققوا

يقبل القسمة هنا، ولا يقبلها هناك

(9) Write a C++ program that displays all the numbers from 300 to 400, 5 per line, that are divisible by 3 but not divisible by 6. Numbers are separated by exactly one space.

هو بيسأل عن إيه؟

داخل المجال المعطى، اختَر مضاعفات ثلاثة، واستبعد مضاعفات ستة. المجال من 300 إلى 400. اطبع خمس قيم في كل سطر، وافصلها بمسافة واحدة.

الفكرة قبل الكود

«لكن لا» ما بتحولش الشرط كله إلى نفي. هي تضيف شرطًا ثانيًا: باقي القسمة الأول صفر، والثاني ليس صفرًا. اختبر كل جزء وحده قبل أن تجمعهما.

نجاح شرط لا يكفي

توقّع قبول المرشح قبل تحريك الاختيار. كل البواقي والحالات هنا من برنامج C++ مشغّل.

303

i % 6 3 المطلوب ≠ 0	&&	i % 3 0 المطلوب = 0

مقبول: الشرطان متحققان. زوّد عدّاد المطبوع.

عدد ما طُبع في هذا التتبّع: **1**

303	·	·	·	·

التتبّع جزء من المجال لتشوف القرار. التشغيل الكامل موجود تحت الكود. السطر الجديد يتبع عدّاد المطبوع، لا قيمة المرشح.

C++ اللي السؤال محتاجها

اكتب `(i % 3 == 0) && (i % 6 != 0)` . بعد القبول استخدم نفس آلية عدّ المطبوع والفصل من Q8. إعادة استخدام الفكرة لا تعني نسخ الشرط من غير قراءة السؤال.

ابنِ الكود جزءًا جزءًا

ابدأ المجال وعدّاد الطباعة

حدود المجال تحدد المرشحين، و`numbers_per_line` يتابع المقبولين.

```
#include<iostream>
using namespace std;

int main() {
    int numbers_per_line = 0;
```

اقبل الأول وارفض الثاني

انتبه لـ!= في الاختبار الثاني. دي ترجمة كلمة not.

```
for (int i = 300; i <= 400; ++i)
    {
        if ((i % 3 == 0) && (i % 6 != 0))
```

اطبع المقبول في مكانه

الفاصل والصف الجديد يعتمدان على عدّاد المطبوع بعد القبول، لا على i.

```
{

            if (numbers_per_line % 5 != 0) cout << " ";
            cout << i;
            numbers_per_line++;
            if (numbers_per_line % 5 == 0) cout << endl;
        }
    }
    if (numbers_per_line % 5 != 0) cout << endl;
    return 0;
}
```

طريقة أخرى لنفس السؤال

في هذا المجال الموجب، الشرط المكافئ هو `i % 6 == 3`. الأعداد المطلوبة تقع بين مضاعفات ستة المتتالية. الصيغة دي اختُبرت على المجال كله؛ لا تنقلها لأعداد سالبة من غير مراجعة سلوك الباقي.

```
#include<iostream>
using namespace std;

int main() {
    int numbers_per_line = 0;

    for (int i = 300; i <= 400; ++i)
    {
        if (i % 6 == 3)
        {

            if (numbers_per_line % 5 != 0) cout << " ";
            cout << i;
            numbers_per_line++;
            if (numbers_per_line % 5 == 0) cout << endl;
        }
    }
    if (numbers_per_line % 5 != 0) cout << endl;
    return 0;
}
```

الطريقة البديلة اتجمّعت واتشغّلت على حالات الاختبار نفسها؛ الناتج المعروض مطابق.

شوف ناتج التشغيل الفعلي

المدخلات المرسلة إلى البرنامج

```
لا يوجد إدخال
```

stdout — رسائل البرنامج والنتائج، بدون صدى لوحة المفاتيح

```
303 309 315 321 327
333 339 345 351 357
```

```
363 369 375 381 387
393 399
```

ممكن تقع في ده

- استخدام == 0 في الاختبار الثاني يختار القيم المطلوب استبعادها.
- OR يسمح لقيم لا تحقق الشرط الأول بالدخول.
- ترك مسافة بعد آخر قيمة من السطر الجزئي يغيّر التنسيق المطلوب.
- نسخ حدود Q8 إلى Q9 يغير مجموعة المرشحين كلها.

جرّب تحسمها بنفسك

الشرط الثاني بيطلب باقي القسمة على ستة...

الإجابة: لا يساوي صفرًا — بالضبط؛ مع بقاء الشرط الأول صحيحًا.

> **لما تقارن بالمصدر:** شرط المرجع ومجاله صحيحان. النسخة هنا تزيل الفاصل الزائد في نهاية السطر الأخير وتضيف نهاية سطر واضحة.

10

ترميز ASCII

الرقم الذي يمثل حرفًا

(10) Write a C++ program that receives an ASCII code (an integer between 0 and 127) and displays its character.
A sample run gives the following output;

```
Enter an ASCII code (0-127): 69
The character for ASCII code 69 is: E
```

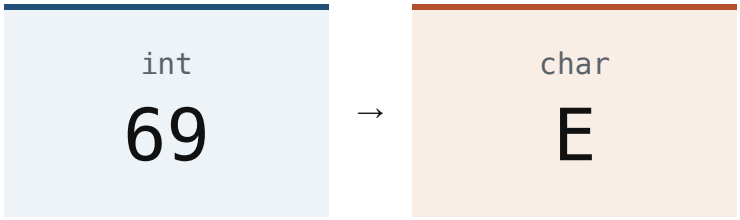
هو بيسأل عن إيه؟

ادخل كود ASCII كعدد صحيح داخل المجال المكتوب، واعرض الحرف الذي يمثله. طباعة نفس العدد مرة ثانية لا تجيب المطلوب. المجال المسموح من 0 إلى 127.

الفكرة قبل الكود

الترميز اتفاق يربط رقمًا بحرف أو بإشارة تحكم. الرقم المكتوب بعلاماته العشرية شيء، والحرف الذي يمثله هذا الكود شيء آخر. تغيير نوع القيمة المعروضة من int إلى char يجعل cout يعرضها كحرف.

اختَر كودًا وشوف معنى التحويل



كود لحرف ظاهر. العدد هو المدخل، والحرف هو التحويل.

```
static_cast<char>(ASCII_Code)
```

اتجاه السهم مهم: المدخل كود عددي. التحويل يطلب تفسيره كحرف. بعض الأكواد لها فعل تحكم بدل شكل مطبوع.

C++ اللي السؤال محتاجها

اقرأ int، افحص المجال، ثم استخدم `static_cast<char>(ASCII_Code)` . النوع داخل cast هو النوع المطلوب. لا تحوّل المدخل إلى string؛ هذا ليس سؤال كلمات أو نص طويل.

ابنِ الكود جزءًا جزءًا

اقرأ الكود وافحصه

التحقق من المجال يمنع تقديم قيمة خارج عقد السؤال كأنها ASCII صالح.

```
#include <iostream>
using namespace std;

int main() {
    int ASCII_Code;
    char character;

    cout << "Enter an ASCII code (0–127): ";
    if (!(cin >> ASCII_Code)) return 0;

    if (ASCII_Code < 0 || ASCII_Code > 127)
```

حوّل طريقة التمثيل

المتغير الجديد char، ولذلك طباعة character تختلف عن طباعة ASCII_Code.

```
{
        cout << "Invalid ASCII code." << endl;
        return 0;
    }
    character = static_cast<char>(ASCII_Code);
```

اعرض الحرف مع الكود

العنوان يوضح أنك تعرض الحرف المقابل للكود، لا أن الاثنين نفس شكل النص.

```
cout << "The character for ASCII code "
         << ASCII_Code << " is: " << character << endl;

    return 0;
}
```

طريقة أخرى لنفس السؤال

الصيغة `char(ASCII_Code)` تؤدي نفس التحويل هنا. static_cast يكتب التحويل بشكل صريح، وfunctional cast أقصر؛ كلتا الصيغتين اختُبرتا.

```
#include <iostream>
using namespace std;

int main() {
    int ASCII_Code;
    char character;

    cout << "Enter an ASCII code (0–127): ";
    if (!(cin >> ASCII_Code)) return 0;

    if (ASCII_Code < 0 || ASCII_Code > 127)
    {
        cout << "Invalid ASCII code." << endl;
        return 0;
    }
    character = char(ASCII_Code);
    cout << "The character for ASCII code "
         << ASCII_Code << " is: " << character << endl;

    return 0;
}
```

الطريقة البديلة اتجمّعت واتشغّلت على حالات الاختبار نفسها؛ الناتج المعروض مطابق.

شوف ناتج التشغيل الفعلي

المدخلات المرسلة إلى البرنامج

```
69
```

```
Enter an ASCII code (0–127): The character for ASCII code 69 is: E
```

ممكن تقع في ده

- `cout << ASCII_Code` يعرض العدد، لا الحرف المطلوب.
- بعض أكواد ASCII لا ترسم حرفًا ظاهرًا. السطر الذي يبدو فارغًا ليس دليلًا كافيًا على فشل التحويل.
- إدخال حرف مكان int يجعل قراءة العدد تفشل؛ استخدم نوع الإدخال المطلوب.
- هذا المثال لا يفسر حروف Unicode أو العربية.

جرّب تحسمها بنفسك

هل كل كود ASCII صالح لازم يرسم حرفًا مرئيًا؟

الإجابة: لا، فيه أكواد تحكم — لذلك فحص بعض الحالات يعتمد على البايت الناتج، مش شكله فقط.

> **لما تقارن بالمصدر:** المثال المعتاد يطابق السؤال. أضيف فحص حدود المجال. أكواد التحكم تظهر في عرض الاختبار بوصف توضيحي داخل أقواس كي تقدر تلاحظ وجودها؛ الأقواس ليست نص البرنامج الأصلي.

11

التمثيل، لا قيمة الرقم

الحرف الذي له كود رقمي

(11) Write a program that receives a character and displays its ASCII code.
A sample run gives the following output;

```
Enter a character: D
ASCII Value of D is 68
```

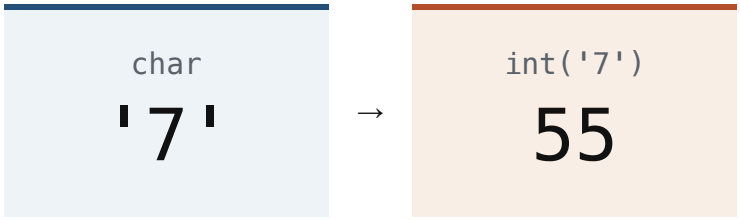
هو بيسأل عن إيه؟

السؤال يعكس الاتجاه السابق: ادخل حرفًا واحدًا، واطبع كوده كعدد. حتى لو دخلت شكل رقم، فهو حرف لأن متغير الإدخال char.

الفكرة قبل الكود

الحرف الرقمي له كود في الترميز. تحويل `'7'` إلى int يظهر كود الحرف، بينما استخراج قيمته كرقم يحتاج عملية أخرى. علامات التنصيص المفردة تغير المعنى: literal للحرف وليس عددًا للحساب.

الحرف '7' له سؤالان مختلفان




```
'7' - '0' = 7
```

الأول كود الحرف، وده المطلوب في السؤال. الثاني قيمته الحسابية كرقم، ودي عملية مختلفة. النتائج المعروضة من تجربة C++ مشغّلة.

تقدر تدخل D وتجرب التحويل العكسي لـQ10: المثال المسجّل يعرض كوده 68. لا تغيّر char إلى int عند القراءة، وإلا تغيّر السؤال نفسه.

C++ اللي السؤال محتاجها

`cin >> c` يقرأ حرفًا غير whitespace. `cout << c` يعرضه كحرف، و`cout << int(c)` يعرض الكود. النوع يختار طريقة العرض.

ابنِ الكود جزءًا جزءًا

اقرأ char

لو أدخلت شكل رقم، سيظل c من نوع char.

```
#include <iostream>
using namespace std;

int main() {
    char c;
    cout << "Enter a character: ";
    if (!(cin >> c)) return 0;
```

اطبع تمثيله العددي

int(c) يطلب عرض الكود العددي، لا إعادة طباعة الحرف.

```
cout << "ASCII Value of " << c << " is " << int(c) << endl;
    return 0;
}
```

طريقة أخرى لنفس السؤال

اكتب `static_cast<int>(c)` بدل `int(c)` . المعنى والنتيجة نفسها في الحالات المسجلة. اختَر الشكل الذي تفهمه مع الحفاظ على اتجاه التحويل.

```
#include <iostream>
using namespace std;

int main() {
    char c;
    cout << "Enter a character: ";
    if (!(cin >> c)) return 0;

    cout << "ASCII Value of " << c << " is " << static_cast<int>(c) << endl;
    return 0;
}
```

الطريقة البديلة اتجمّعت واتشغّلت على حالات الاختبار نفسها؛ الناتج المعروض مطابق.

شوف ناتج التشغيل الفعلي

المدخلات المرسلة إلى البرنامج

```
D
```

stdout — رسائل البرنامج والنتائج، بدون صدى لوحة المفاتيح

```
Enter a character: ASCII Value of D is 68
```

ممكن تقع في ده

- تعريف c كـint يغيّر ما تحاول cin قراءته.
- طباعة c من غير cast تجيب الحرف نفسه.
- اعتبار كود الحرف الرقمي قيمته الحسابية يخلط التمثيل بالحساب.
- cin >> char يتخطى المسافات؛ المثال لا يقرأ سطرًا كاملًا.

جرّب تحسمها بنفسك

int(c) لو c حرف رقمي بيرجع...

الإجابة: كود الحرف — التحويل يعرض التمثيل العددي للحرف.

لما تقارن بالمصدر: الناتج يطابق المثال، مع إضافة نهاية سطر. المدخل هنا حرف ASCII واحد غير فارغ، لا نص متعدد الحروف.

12

حروف كبيرة وصغيرة

اختبر حالة الحرف ثم حوّلها

(12) Write a C++ program that prompts the user to enter an uppercase letter and converts it to a lowercase letter.

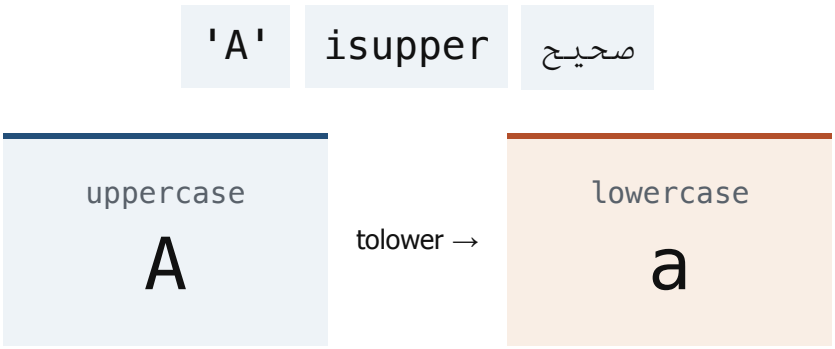
هو بيسأل عن إيه؟

اطلب حرف uppercase وحوّله إلى lowercase. لو المدخل ليس uppercase، وضّح إنه لا يطابق المدخل المطلوب بدل معاملته كحالة صحيحة.

الفكرة قبل الكود

اختبار الحرف وتحويله عمليتان مختلفتان. isupper يسأل هل الحرف كبير، وtolower ينتج الحرف الصغير المقابل. تغيير حالة الحرف لا يعني تغيير اسم المتغير ولا نوعه.

قرار، وبعده تحويل



فحص isupper لا يغير A. ناتج tolower هو التحويل، وتخزينه في char يجعل cout يعرض a.

```
int result → 97   |   char result → a
```

الدالة ترجع int. المثال هنا يوضح فرق طريقة العرض لو طبعت ناتجها مباشرة. نجاح isupper يعني قيمة غير صفرية؛ لا تفترض أنها تساوي قيمة ثابتة بعينها.

C++ اللي السؤال محتاجها

المرجع يستخدم `<cctype>`. بعد قراءة char، افحص `isupper(inputChar)`. داخل الفرع خزّن ناتج tolower في char قبل الطباعة، لأن الدالة ترجع كودًا من نوع int.

ابنِ الكود جزءًا جزءًا

اقرأ الحرف

الإدخال حرف واحد، والنوع لا يتغير أثناء اختبار الفئة.

```
#include <iostream>
#include <cctype>
using namespace std;

int main() {
    char inputChar;

    cout << "Enter an uppercase letter: ";
    if (!(cin >> inputChar)) return 0;
```

اسأل ثم حوّل

isupper لا تعدّل الحرف. tolower هي التي توفر ناتج التحويل.

```
if (isupper(inputChar)) {
        char lowercaseChar = tolower(inputChar);
```

اطبع أو وضّح رفض المدخل

الفرع الآخر يشرح أن المدخل لا يحقق شرط uppercase.

```
cout << "The lowercase letter is: " << lowercaseChar << endl;
    } else {

        cout << "Invalid input. Please enter an uppercase letter." << endl;
    }

    return 0;
}
```

طريقة أخرى لنفس السؤال

في ASCII يمكن فحص المجال بين `'A'` و`'Z'` ، ثم إضافة الفرق `'a' - 'A'` . الحروف هنا تشرح معنى الإزاحة بدل حفظ رقم بلا سبب. هذه الطريقة مرتبطة بعقد ASCII.

```
#include <iostream>
using namespace std;

int main() {
    char inputChar;

    cout << "Enter an uppercase letter: ";
    if (!(cin >> inputChar)) return 0;

    if (inputChar >= 'A' && inputChar <= 'Z') {
        char lowercaseChar = char(inputChar + ('a' - 'A'));
        cout << "The lowercase letter is: " << lowercaseChar << endl;
    } else {

        cout << "Invalid input. Please enter an uppercase letter." << endl;
    }

    return 0;
}
```

الطريقة البديلة اتجمّعت واتشغّلت على حالات الاختبار نفسها؛ الناتج المعروض مطابق.

شوف ناتج التشغيل الفعلي

المدخلات المرسلة إلى البرنامج

```
A
```

stdout — رسائل البرنامج والنتائج، بدون صدى لوحة المفاتيح

```
Enter an uppercase letter: The lowercase letter is: a
```

ممكن تقع في ده

- استدعاء isupper وحده لا يحوّل شيئًا.
- طباعة tolower مباشرةً قد تعرض كودًا عدديًا؛ خزّن النتيجة في char كما في المرجع.
- قبول lowercase لأن tolower تتركه كما هو يتجاهل شرط السؤال عن المدخل.
- لا تعمم إزاحة ASCII على أي لغة أو ترميز.

جرّب تحسمها بنفسك

isupper وظيفتها...

الإجابة: اختبار فئة الحرف — النتيجة غير الصفرية تعني أن الاختبار نجح.

> **لما تقارن بالمصدر:** الدوال متفقة مع حل المرجع. cctype مشروحة هنا لأن الحل يستخدمها، والمثال مقصور على حروف ASCII.

تصنيف وشروط متتابعة

13

اختيار رسالة واحدة من فئة الحرف

(13) Design a C++ program that prompts the user to input a single character, which can be either:

- A letter (uppercase or lowercase), or
- A single-digit number (0-9).

The program should then read the input and display one of the following messages to the user, based on the character type:

- "Uppercase Letter" if the character is an uppercase letter (A-Z).
- "Lowercase Letter" if the character is a lowercase letter (a-z).
- "Number" if the character is a single-digit number (0-9).
- "Invalid Input" if the input is not a letter or a single-digit number.

هو بيسأل عن إيه؟

اقرأ حرفًا واحدًا، واطبع الرسالة المطلوبة حسب كونه uppercase أو lowercase أو digit. لو مش واحدًا منهم، اطبع Invalid Input.

الفكرة قبل الكود

كل فئة لها حدود أو اختبار يميزها. حرف الرقم يعتبر digit حتى لو شكله يشبه عددًا نستخدمه في الحساب. هذا السؤال يصنّف الشكل المدخل، ولا يحسب به ولا يستخدمه كإشارة توقف.

نفس المدخل يمشي في مسار واحد

A

isupper	islower	isdigit	else
Uppercase	Lowercase	Number	Invalid

Uppercase Letter

آخر else لا يعني فشل البرنامج؛ هو الإجابة المناسبة لمدخل مثل علامة الاستفهام. اطلب النوع الصحيح من char، ثم اختبر فئته.

C++ اللي السؤال محتاجها

سلسلة `if / else if / else` تجعل مسار القرار واضحًا. اختبر isupper ثم islower ثم isdigit، والـelse الأخيرة تجمع الحالات غير المقبولة. حافظ على نص الرسائل المطلوب.

ابنِ الكود جزءًا جزءًا

اقرأ حرفًا واحدًا

cin تقرأ char واحدًا؛ لا تختبر أن كل النص المكتوب حرف واحد.

```
#include <iostream>
#include <cctype>
using namespace std;

int main() {
    char inputChar;

    cout << "Enter a single character (uppercase or lowercase letter or a digit 0:9): ";
    if (!(cin >> inputChar)) return 0;
```

مرّ على الفئات بالترتيب

بعد نجاح فرع، لا ينفذ باقي السلسلة. else الأخيرة تتعامل مع الرموز غير التابعة للفئات.

```
if (isupper(inputChar)) cout << "Uppercase Letter" << endl;
    else if (islower(inputChar)) cout << "Lowercase Letter" << endl;
    else if (isdigit(inputChar)) cout << "Number" << endl;
    else cout << "Invalid Input" << endl;

    return 0;
}
```

طريقة أخرى لنفس السؤال

قارن مباشرةً بمجالات ASCII: من 'A' إلى 'Z' ، ومن 'a' إلى 'z' ، ومن '0' إلى '9' . علامات التنصيص مهمة لأنها حدود حروف.

```
#include <iostream>
using namespace std;

int main() {
    char inputChar;

    cout << "Enter a single character (uppercase or lowercase letter or a digit 0:9): ";
    if (!(cin >> inputChar)) return 0;

    if (inputChar >= 'A' && inputChar <= 'Z') cout << "Uppercase Letter" << endl;
    else if (inputChar >= 'a' && inputChar <= 'z') cout << "Lowercase Letter" << endl;
    else if (inputChar >= '0' && inputChar <= '9') cout << "Number" << endl;
    else cout << "Invalid Input" << endl;

    return 0;
}
```

الطريقة البديلة اتجمّعت واتشغّلت على حالات الاختبار نفسها؛ الناتج المعروض مطابق.

شوف ناتج التشغيل الفعلي

المدخلات المرسلة إلى البرنامج

```
A
```

stdout — رسائل البرنامج والنتائج، بدون صدى لوحة المفاتيح

```
Enter a single character (uppercase or lowercase letter or a digit 0:9): Uppercase Letter
```

ممكن تقع في ده

- كتابة حدود digit كالأعداد 0 و9 بدل الحرفين '0' و'9' تختبر مجال أكواد مختلفًا.
- if مستقلة ثم else مع آخر if فقط قد تعطي رسالة إضافية غير مقصودة.
- إدخال كلمة لا يحول البرنامج لمصنّف كلمات؛ الباقي يظل غير مقروء.
- الحرف '0' هنا Number؛ مش إشارة توقف Q1.

جرّب تحسمها بنفسك

الحرف '0' في هذا السؤال...

الإجابة: Number — لأننا نصنّف digit character.

> **لما تقارن بالمصدر:** الفئات والرسائل تتفق مع السؤال والمرجع. الاختبار يغطي الفروع المختلفة ضمن مدخل حرف ASCII واحد غير whitespace.

14

تتابع ASCII وتنسيق صفوف

من الأكواد إلى صفوف حروف

(14) Write a C++ program that prints all the uppercase characters of the ASCII character table. Display 5 characters per line. Note that ASCII for $A = 65$ and $Z = 90$.

هو بيسأل عن إيه؟

اطبع الحروف uppercase كلها. النطاق المعطى يشمل أول حرف وآخر حرف، والسطر الأخير مسموح يكون غير مكتمل. الحروف من A إلى Z، بخمسة حروف في كل سطر ومسافة واحدة بينها.

الفكرة قبل الكود

في نطاق ASCII الذي يحدده السؤال، أكواد الحروف الكبيرة متتابعة. تقدر تمر على الكود كعدد، ثم تعرضه كحرف. آلية تقسيم الصفوف لا تتغير لو المحتوى انتقل من أرقام إلى حروف.

الصف يحسب حروفًا، لا أكوادًا

`65 → A` … `90 → Z`

A	B	C	D	E
F	G	H	I	J
K	L	M	N	O
P	Q	R	S	T
U	V	W	X	Y
Z				

لاحظ آخر صف: Z وحده ما زال يحتاج نهاية سطر. عدّاد الحروف يزيد مرة لكل حرف يُطبع. معامل % يحدد متى اكتملت أماكن الصف.

C++ اللي السؤال محتاجها

النسخة الأساسية تستخدم عدّادًا عدديًا وتحوله إلى char وقت الطباعة. عدّاد آخر يتابع عدد الحروف المطبوعة. أضف المسافة بين الحروف، ونهاية السطر بعد المجموعة أو بعد الباقي الأخير.

ابنِ الكود جزءًا جزءًا

جهّز العدّاد

`char_per_line` يتابع مواضع الحروف على الشاشة، لا أكوادها.

```
#include<iostream>

using namespace std;

int main() {
    int char_per_line = 0;
```

كرر التحويل والطباعة

المجال شامل للحرف الأخير. التحويل يحدث لكل كود، ثم تتعامل قاعدة التنسيق مع الحرف المطبوع.

```
for (int i = 65; i <= 90; ++i)
    {

        if (char_per_line % 5 != 0) cout << " ";
        cout << static_cast<char>(i);
        char_per_line++;

        if (char_per_line % 5 == 0) cout << endl;
    }

    if (char_per_line % 5 != 0) cout << endl;
    return 0;
}
```

طريقة أخرى لنفس السؤال

يمكن أن يكون عدّاد التكرار نفسه char يبدأ بـ 'A' وينتهي بـ 'Z' . عندها لا تحتاج cast وقت العرض. تتابع الحروف هنا مستند إلى ASCII الذي حدده السؤال.

```
#include<iostream>

using namespace std;

int main() {
    int char_per_line = 0;

    for (char letter = 'A'; letter <= 'Z'; ++letter)
    {

        if (char_per_line % 5 != 0) cout << " ";
        cout << letter;
        char_per_line++;

        if (char_per_line % 5 == 0) cout << endl;
    }

    if (char_per_line % 5 != 0) cout << endl;
    return 0;
}
```

الطريقة البديلة اتجمّعت واتشغّلت على حالات الاختبار نفسها؛ الناتج المعروض مطابق.

شوف ناتج التشغيل الفعلي

المدخلات المرسلة إلى البرنامج

```
لا يوجد إدخال
```

stdout — رسائل البرنامج والنتائج، بدون صدى لوحة المفاتيح

```
A B C D E
F G H I J
K L M N O
P Q R S T
U V W X Y
Z
```

ممكن تقع في ده

- استخدام < بدل <= عند النهاية يسقط الحرف الأخير.
- حذف السطر الجزئي يترك جزءًا من الحروف من غير طباعة.
- الاعتماد على قيمة الكود لتقسيم كل خمس حروف يخلط الكود بموضع الإخراج.
- تكرار الحروف لملء السطر الأخير يغير المطلوب.

جرّب تحسمها بنفسك

لو آخر مجموعة مش كاملة...

الإجابة: نطبعها كما هي وننهي السطر — كل الحروف المطلوبة تفضل موجودة.

> **لما تقارن بالمصدر:** المرجع يطبع نفس الحروف، لكن يترك مسافة بعد الحرف الأخير دون نهاية سطر. النسخة هنا تختم السطر بشكل واضح وتستخدم مسافات بين الحروف فقط.

15

ضرب وتكرار

مضاعفات العدد، مش قواه

(15) Write a C++ program that reads an integer and displays its first five multiples. That is in the input number is N the program displays $\times 1$, $N \times 2$, $N \times 3$, $N \times 4$, $N \times 5$

هو بيسأل عن إيه؟

ادخل عددًا N، واطبع أول خمسة مضاعفات له. N ثابت في كل العمليات؛ الذي يتحرك هو المضاعف الذي نضربه فيه.

الفكرة قبل الكود

المضاعفات هي نفس العدد مأخوذًا عدد مرات مختلفًا. ضرب الناتج السابق في N كل مرة يصنع قوى، لا المضاعفات المطلوبة. الصفر والسالب لهما مضاعفات أيضًا؛ المجال ليس للأعداد الموجبة فقط.

المدخل ثابت، والمضاعِف بيتحرك

7

7	14	21	28	35

$$N \times i$$

التكرار يغيّر i. لو غيّرت N نفسه كل مرة، فقدت العدد الأصلي. لاحظ أن الصفر والعدد السالب لا يحتاجان قانونًا جديدًا.

C++ اللي السؤال محتاجها

اقرأ N قبل for، ثم احسب `N * i` داخلها. افصل الحساب عن الفواصل: `if (i < 5)` تطبع الفاصلة فقط لو فيه عنصر بعد الحالي.

ابنِ الكود جزءًا جزءًا

احتفظ بالعدد الأصلي

لا تغيّر N أثناء بناء القائمة. كل حاصل ضرب يرجع لنفس القيمة.

```
#include<iostream>

using namespace std;

int main() {
    int N;

    cout << "Enter an integer: ";
    if (!(cin >> N)) return 0;
```

احسب واضبط الفاصل

i يتغير من أول مضاعف إلى آخره. الاختبار الداخلي يمنع فاصلة زائدة بعد النهاية.

```
cout << "The first five multiples of " << N << " are: ";

    for (int i = 1; i <= 5; ++i)
    {
        cout << (N * i);
        if (i < 5) cout << ", ";
    }
    cout << endl;
    return 0;
}
```

طريقة أخرى لنفس السؤال

while تؤدي نفس المهمة لو وضعت تهيئة i قبلها والزيادة داخلها. نفس المدخل الثابت، ونفس المضاعفات والفواصل.

```
#include<iostream>

using namespace std;

int main() {
    int N;

    cout << "Enter an integer: ";
    if (!(cin >> N)) return 0;

    cout << "The first five multiples of " << N << " are: ";

    int i = 1;
    while (i <= 5)
    {
        cout << (N * i);
        if (i < 5) cout << ", ";
        i++;
    }
    cout << endl;
    return 0;
}
```

الطريقة البديلة اتجمّعت واتشغّلت على حالات الاختبار نفسها؛ الناتج المعروض مطابق.

شوف ناتج التشغيل الفعلي

المدخلات المرسلة إلى البرنامج

```
7
```

stdout — رسائل البرنامج والنتائج، بدون صدى لوحة المفاتيح

```
Enter an integer: The first five multiples of 7 are: 7, 14, 21, 28, 35
```

ممكن تقع في ده

- تغيير N إلى N+1 يعطي أعدادًا متتالية، لا مضاعفات.
- ضرب الناتج السابق في N يعطي قوى.
- بدء i من الصفر يضيف حاصلًا غير موجود ضمن أول المضاعفات المطلوبة.
- طباعة فاصلة بعد كل حاصل تترك فاصلة أخيرة زائدة.
- القيم الكبيرة جدًا قد تعمل overflow. لازم N والمضاعفات المطلوبة كلها تكون داخل مدى int المستخدم.

جرّب تحسمها بنفسك

أي قيمة تفضل ثابتة أثناء بناء القائمة؟

الإجابة: N المدخل — كل عملية تستخدم نفس N.

> **لما تقارن بالمصدر:** الحد الأول في الصيغة المطبوعة ناقصه N، لكن نص السؤال وحل المرجع يوضحان المقصود. النسخة تحسب المضاعفات كما في المرجع.

16

أنماط وصفوف وتكرار متداخل

ابنِ سطر الهرم، ثم كرره

(16) Write a C++ program that prompts the user to enter an integer from 1 to 15 and displays a pyramid like that one shown in the below figure.

```
                1
              2 1 2
            3 2 1 2 3
          4 3 2 1 2 3 4
        5 4 3 2 1 2 3 4 5
      6 5 4 3 2 1 2 3 4 5 6
    7 6 5 4 3 2 1 2 3 4 5 6 7
```

هو بيسأل عن إيه؟

اقرأ ارتفاعًا داخل المجال المطلوب. كل سطر ينزل من رقم السطر إلى الوسط، ثم يصعد مرة أخرى، مع فراغات تضبط شكل الهرم. حجم المثال المصور ليس حجمًا ثابتًا لكل تشغيل. الارتفاع N من 1 إلى 15.

الفكرة قبل الكود

قسّم السطر لثلاث مناطق: فراغات في البداية، نصف هابط يشمل الوسط، ونصف صاعد يبدأ بعد الوسط. التكرار الخارجي يختار السطر؛ التكرارات الداخلية تنفذ شغل السطر. هذا امتداد للتكرار المتداخل بعد أشكال الحلقة الأساسية في المحاضرة.

ابنِ صفًا قبل ما تبني الهرم

اختَر صفًا من تشغيل المثال. الفراغات الأولى تظهر بنقط لتقدر تعدّ مكانها؛ هذه النقط ليست من ناتج البرنامج.

```
........... 3 2 1 2 3
```

رمادي: فراغات البداية أزرق: نزول حتى المركز برتقالي: صعود بعد المركز

الرمادي يمثّل الفراغات الأولى فقط. الأزرق يطبع حتى المركز، والبرتقالي يبدأ بعد المركز.

حقل الرقم عرضه ثابت، وكذلك مجموعة الفراغات. ده يحافظ على المحاذاة لما الرقم يتكوّن من أكثر من خانة. التشغيل الكامل يحافظ على الفراغات الأصلية.

C++ اللي السؤال محتاجها

الفراغات تتبع الفرق بين ارتفاع الهرم ورقم السطر. الطباعة الرقمية تستخدم `setw(3)`، والفراغات الابتدائية تستخدم نفس عرض الخانة. `endl` يأتي بعد اكتمال الأجزاء الثلاثة، لا داخل واحد منها.

ابنِ الكود جزءًا جزءًا

اقرأ الحجم وافحص مجاله

نرفض مجالًا غير مناسب قبل الدخول في بناء الصفوف.

```
#include<iostream>
#include <iomanip>
using namespace std;

int main() {
    int N;

    cout << "Enter an integer from 1 to 15: ";
    if (!(cin >> N)) return 0;
    if (N < 1 || N > 15)
```

ابدأ السطر واضبط موقعه

كل سطر أقصر يحتاج فراغات أكثر على يساره ليظل الوسط ثابتًا.

```
{
        cerr << "Invalid input. Please enter an integer between 1 and 15." << endl;
        return 0;
    }

    for (int i = 1; i <= N; ++i)
    {

        for (int j = 0; j < N - i; ++j) cout << "   ";
```

ابنِ النصف الهابط

هذا الجزء يطبع الوسط أيضًا، فلا تكرره في الجزء التالي.

```
for (int k = i; k >= 1; --k) cout << setw(3) << k;
```

أكمل الصعود وأنهِ السطر

الجزء الصاعد يبدأ بعد الوسط. بعده فقط نطبع endl وندخل سطرًا جديدًا.

```
for (int l = 2; l <= i; ++l) cout << setw(3) << l;

        cout << endl;
    }
    return 0;
}
```

طريقة أخرى لنفس السؤال

أعد كتابة التكرارات بـwhile مع تهيئة كل عدّاد في مكانه. المهم إعادة تهيئة العدادات الداخلية لكل سطر. الشكل الكامل والمسافات، بما فيها المدخل الأقصى، طابقت النسخة الأساسية.

```
#include<iostream>
#include <iomanip>
using namespace std;

int main() {
    int N;

    cout << "Enter an integer from 1 to 15: ";
    if (!(cin >> N)) return 0;
    if (N < 1 || N > 15)
    {
        cerr << "Invalid input. Please enter an integer between 1 and 15." << endl;
        return 0;
    }

    int i = 1;
    while (i <= N)
    {
        int j = 0;
        while (j < N - i)
        {
            cout << "   ";
            j++;
        }

        int k = i;
        while (k >= 1)
        {
            cout << setw(3) << k;
            k--;
        }

        int l = 2;
        while (l <= i)
        {
            cout << setw(3) << l;
            l++;
        }

        cout << endl;
        i++;
    }
    return 0;
}
```

الطريقة البديلة اتجمّعت واتشغّلت على حالات الاختبار نفسها؛ الناتج المعروض مطابق.

شوف ناتج التشغيل الفعلي

المدخلات المرسلة إلى البرنامج

```
7
```

stdout — رسائل البرنامج والنتائج، بدون صدى لوحة المفاتيح

```
Enter an integer from 1 to 15:                  1
                2  1  2
             3  2  1  2  3
          4  3  2  1  2  3  4
       5  4  3  2  1  2  3  4  5
    6  5  4  3  2  1  2  3  4  5  6
 7  6  5  4  3  2  1  2  3  4  5  6  7
```

ممكن تقع في ده

- بدء النصف الصاعد من الوسط يطبع الوسط مرتين.
- نسيان إعادة عدّاد داخلي يجعله يحتفظ بنهاية السطر السابق.
- وضع endl داخل تكرار الأرقام يحوّل السطر إلى عمود.
- مسافة واحدة بين الخانات لا تعالج محاذاة الأرقام التي لها أكثر من رقم. setw يحجز حدًا أدنى ولا يقطع العدد.

جرّب تحسمها بنفسك

ليه النصف الصاعد لا يبدأ من الوسط؟

الإجابة: لأن النصف الهابط طبعه بالفعل — كل سطر له وسط واحد فقط.

> **لما تقارن بالمصدر:** شكل الهرم الصحيح متفق مع المرجع. تم فحص الحدين وحجم المثال والحجم الأقصى، مع التحقق من المسافات والأرقام في كل سطر.

جرّب الكود على جهازك

افتح ملف C++ جديد، واكتب البرنامج الكامل الخاص بالسؤال. في Dev-C++ استخدم **File** → **New** → **Source File** (`Ctrl+N`)، واحفظ من **File** → **Save As**. للتجميع والتشغيل اختَر **Execute** → **Compile & Run** (`F11`). في VS Code أنشئ الملف من **File** → **New Text File** (`Ctrl+N` على Windows)، ثم احفظه بامتداد `.cpp`. إعداد compiler والتشغيل يعتمد على إعداد جهازك.

Sheet 4 Lecture 6 حفظ ملف C++ — يحتاج إنترنت ارجع لأول سؤال

للتسليم، اجمع ملفاتك في مجلد `XXX_Sheet_4`؛ استبدل XXX برقمك في الكشف مكتوبًا في ثلاث خانات. سمِّ الملفات من `Q_0401.cpp` إلى `Q_0416.cpp`، بلا مسافات أو أسماء عربية. بعد إنهاء الحل، احذف ملفات `.exe` من نسخة التسليم واضغط المجلد في ملف ZIP أو RAR واحد؛ ارفعه على الرابط الذي يُعلَن للتسليم. خطوات ضغط المجلد — تحتاج إنترنت.

الكود المعروض متوافق مع C++98، وكل برنامج وطريقته البديلة اتجمّعوا واتشغّلوا بلا تحذيرات. الأرقام في الأمثلة من تشغيل فعلي. فروق النسخ المشروحة عن نص الشيت أو حل المرجع موضحة عند كل سؤال.